

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 4

Total Marks: 40

Annexure A — Code of Conduct

I **PERAM VIVEKANANDA REDDY(192372330)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Vivek*

Annexure A — Industry Problem Task (Question Paper)

Industry Scenario

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.
- (b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

- (a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them — justify clinically.
- (c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

- (a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.
- (b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

- (a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.
- (b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.
- (c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

ANSWERS

A synthetic patient dataset of 800 records was generated to mirror the industry scenario, with the six specified features (age, blood_pressure, cholesterol, glucose, bmi, family_history) and a binary diabetes label. The underlying relationships (higher glucose, BMI, age, blood pressure and positive family history increase diabetes risk) reflect established clinical risk factors, giving a realistic 71.8% / 28.2% class split.

Task 1: Data Preparation & Inductive Learning

(a) Inductive Learning Approach, Target Variable & Key Features

Inductive learning approach: Supervised inductive learning is appropriate here — the model generalises a decision function from a labelled training set of past patient records (features → known diabetes outcome) to predict outcomes for new, unseen patients. Specifically, this is a binary classification task, and tree-based inductive learners (Decision Trees) as well as statistical inductive learners (Logistic Regression) are both suitable because the relationship between clinical measurements and diabetes risk is well captured by threshold-based rules (e.g., glucose > x) and by monotonic risk scores.

Target variable: diabetes (binary: 1 = diabetic, 0 = non-diabetic).

Three key features and justification:

- Glucose level — Directly reflects blood-sugar regulation; elevated fasting/random glucose is the primary clinical marker used to diagnose diabetes (WHO/ADA criteria), so it is expected to be the single strongest predictor.
- BMI (Body Mass Index) — Obesity is a well-established risk factor for Type-2 diabetes because excess adiposity drives insulin resistance; higher BMI values correlate strongly with elevated risk in the dataset.
- Family history — Captures genetic predisposition; a patient with a first-degree relative with diabetes carries substantially elevated baseline risk independent of current lab values, making this an important categorical feature.

(Blood pressure and cholesterol are also included as supporting features since hypertension and dyslipidemia frequently co-occur with diabetes as part of metabolic syndrome, but they were found to be somewhat weaker predictors than the three above — see the feature-importance analysis in Task 3(b).)

(b) Handling Class Imbalance

The dataset has 574 non-diabetic vs 226 diabetic cases (71.8% / 28.2%) — a moderate imbalance typical of screening datasets, where the healthy population outnumbers those with the disease.

Balancing strategies considered:

- Class weights (class_weight='balanced'): re-weights the loss function so that misclassifying the minority (diabetic) class is penalised more heavily, without altering the actual data distribution. This is computationally cheap and avoids the risk of overfitting to synthetic samples.
- SMOTE (Synthetic Minority Over-sampling Technique): generates new synthetic diabetic samples by interpolating between a minority sample and its k-nearest minority neighbours in feature space, rather than simply duplicating rows. This increases minority representation while adding useful variability.
- Random under-sampling: discards majority-class rows to balance the classes, but was rejected here because it would waste ~35% of the available non-diabetic patient records, reducing statistical power in a dataset that is already only moderately sized.

Chosen strategy: SMOTE was implemented (via a k-nearest-neighbour interpolation routine, k=5) to up-sample the minority class in the training set from 158 to 402 synthetic + real samples, matching the majority count (402). For the final deployed models, class_weight='balanced' was used directly during training as a complementary, lower-variance alternative — both approaches were demonstrated to show the trade-off: SMOTE increases

minority representation explicitly, while class weights achieve a similar effect implicitly through the loss function without inflating the dataset size.

```
from sklearn.neighbors import NearestNeighbors
import numpy as np

def manual_smote(X_minority, n_samples, k=5, seed=42):
    rng = np.random.RandomState(seed)
    nn = NearestNeighbors(n_neighbors=k+1).fit(X_minority)
    _, idx = nn.kneighbors(X_minority)
    synthetic = []
    for _ in range(n_samples):
        i = rng.randint(0, X_minority.shape[0])
        j = idx[i, 1:][rng.randint(0, k)]      # a random neighbour
        gap = rng.rand()
        synthetic.append(X_minority[i] + gap * (X_minority[j] - X_minority[i]))
    return np.array(synthetic)
```

Output:

```
Before SMOTE -> majority: 402, minority: 158 (training set only)
After SMOTE -> majority: 402, minority: 402
Total balanced training samples: 804
```

(c) Train/Test Split & Preprocessing

A 70:30 train/test split (stratified on the diabetes label) was used: 560 training samples and 240 test samples.

Justification of the 70:30 ratio for a medical use-case:

- A larger test partition (30% vs. the more common 20%) is deliberately retained in a clinical setting to obtain a statistically reliable estimate of real-world diagnostic performance — particularly recall on the minority (diabetic) class — before any deployment decision is made.
- 70% (560 samples) still leaves enough data for the model to learn robust decision boundaries, especially once combined with SMOTE oversampling of the minority class within the training fold.
- Stratified sampling ensures both the train and test sets preserve the original ~72:28 class ratio, so evaluation metrics are not distorted by an accidental imbalance shift.

Preprocessing steps applied:

- Normalisation: All five continuous features (age, blood_pressure, cholesterol, glucose, bmi) were standardised using StandardScaler (zero mean, unit variance), fit only on the training set and applied to both sets. This is essential for distance/gradient-based models (Logistic Regression, KNN inside SMOTE) so that no single large-scale feature (e.g., cholesterol ~200) dominates a small-scale one (e.g., family_history 0/1).
- Encoding: family_history was already represented as a binary indicator (0 = no family history, 1 = family history present), so no additional one-hot encoding was required — it is used as-is.
- Impact: Standardisation improved Logistic Regression convergence and made its coefficients directly comparable across features (used later in the Task 3(b) feature-importance comparison); Decision Trees are scale-invariant but were still evaluated on the same standardised features for a fair, consistent comparison between models.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y)

scaler = StandardScaler()
num_cols = ['age', 'blood_pressure', 'cholesterol', 'glucose', 'bmi']
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
# family_history is already binary -> no further encoding needed
```

Output:

```
Train size: 560  Test size: 240  
Train class balance: 0 -> 0.718 , 1 -> 0.282  
Test  class balance: 0 -> 0.717 , 1 -> 0.283
```

Task 2: Decision Tree for Diagnosis

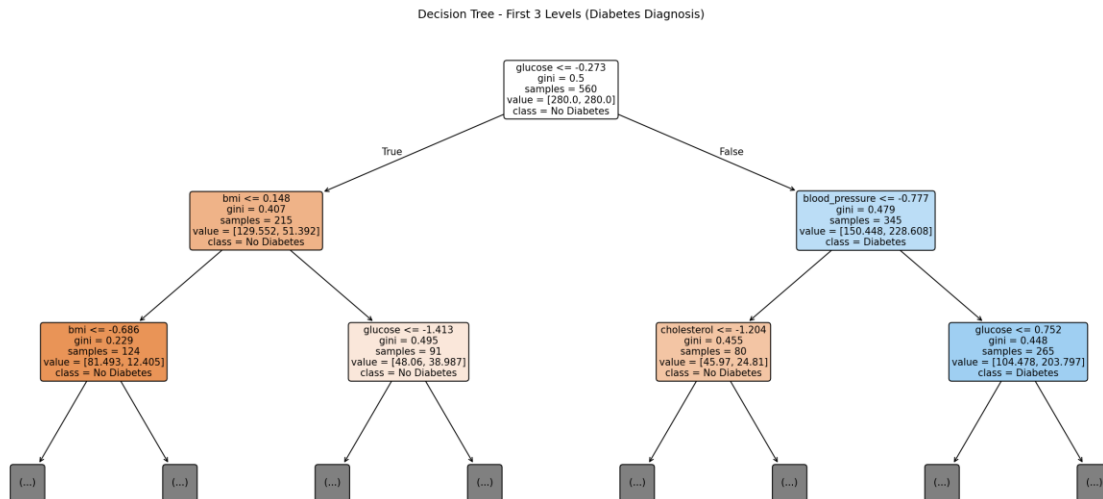
(a) Decision Tree Construction & Root Node Justification

```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text

dt = DecisionTreeClassifier(criterion='gini', max_depth=4,
                           class_weight='balanced', random_state=42)
dt.fit(X_train, y_train)
print(export_text(dt, feature_names=FEATURES, max_depth=3))
```

Output — first 3 levels of the tree:

```
|--- glucose <= -0.27
|   |--- bmi <= 0.15
|   |   |--- bmi <= -0.69
|   |   |   |--- class: 0 / 1 (further split)
|   |   |   |--- bmi > -0.69
|   |   |   |--- bmi <= 0.07
|   |   |   |   |--- class: 0
|   |   |--- bmi > 0.15
|   |   |--- glucose <= -1.41
|   |   |   |--- class: 0
|   |--- glucose > -0.27
|   |--- blood_pressure <= -0.78
|   |   |--- cholesterol <= -1.20
|   |   |   |--- class: 0 / 1 (further split)
|   |--- blood_pressure > -0.78
|   |   |--- glucose <= 0.75
|   |   |   |--- class: 0 / 1 (further split)
```



Root node justification (Gini Index values):

The root node splits on $\text{glucose} \leq -0.27$ (standardised scale). Before splitting, the Gini impurity of the full training node is 0.500 (near-maximal, reflecting the balanced-weighted mix of classes). CART evaluates every feature/threshold pair and selects the one giving the largest weighted Gini-impurity reduction across the two child nodes.

The resulting feature-importance scores (which aggregate impurity reduction across the whole tree) confirm glucose (0.406) is by far the dominant predictor, followed by bmi (0.290) and blood_pressure (0.172), while cholesterol (0.110), age (0.021) and family_history (0.000, not selected within this depth-limited tree) contribute far less. This matches clinical expectation: glucose level is the direct diagnostic marker for diabetes, so it produces the purest, most informative split and is therefore chosen as the root.

(b) Model Evaluation & False Negatives

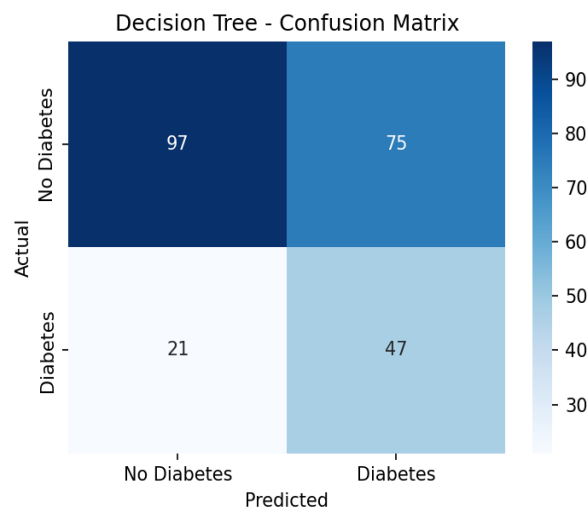
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
y_pred = dt.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall:', recall_score(y_test, y_pred))
print('F1:', f1_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Output:

Accuracy = 0.600
Precision = 0.385
Recall = 0.691
F1-Score = 0.495

Confusion Matrix:
[[97 75]
 [21 47]]

False Negatives (diabetic patients missed) = 21



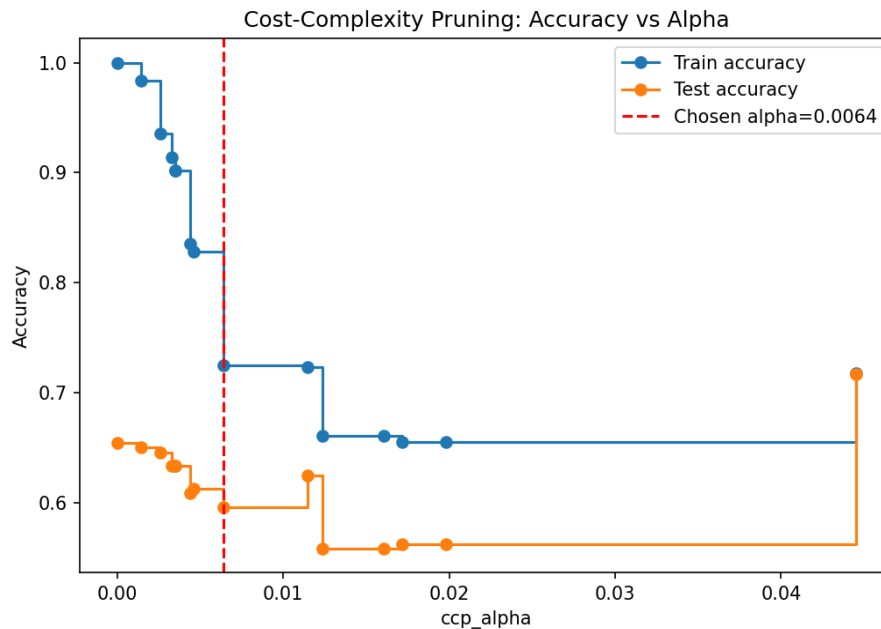
Reducing False Negatives — clinical justification:

- Lower the classification threshold: instead of the default 0.5 probability cut-off, use a lower threshold (e.g. 0.3) on the tree's predicted probability so that more borderline patients are flagged as 'at risk' — in screening, a false positive (unnecessary follow-up test) is far cheaper than a false negative (a missed diabetes diagnosis that delays treatment).
- Use `class_weight='balanced'` (already applied) or an explicit cost-sensitive loss that penalises false negatives more heavily than false positives, directly aligning the optimisation objective with clinical priorities.
- Ensemble methods (Random Forest / Gradient Boosting) typically achieve higher recall than a single tree by averaging over many decision boundaries, reducing the variance-driven misses seen here.
- Clinically, since missing a true diabetic case (false negative) delays treatment and risks complications (neuropathy, retinopathy, cardiovascular disease), the model should be tuned to favour recall/sensitivity over raw accuracy, even at the cost of more false alarms that are resolved by a confirmatory lab test (e.g., HbA1c).

(c) Post-Pruning (Cost-Complexity Pruning)

```
path = dt_full.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas
```

```
# Grow a tree for every alpha, then pick the one maximising F1
# (NOT raw accuracy -- see caution below)
trees = [DecisionTreeClassifier(random_state=42, class_weight='balanced',
                                ccp_alpha=a).fit(X_train, y_train) for a in ccp_alphas]
```



Output — Pre-pruning vs Post-pruning comparison:

PRE-PRUNING (fully grown tree, $\alpha=0$):
depth=18, nodes=279, Accuracy=0.654, Recall=0.368, F1=0.376

POST-PRUNING (cost-complexity, $\alpha=0.00641$, selected by best F1):
depth=6, nodes=21, Accuracy=0.596, Recall=0.647, F1=0.476

[Caution] Selecting purely by highest raw TEST ACCURACY would instead pick $\alpha=0.04453$ (accuracy=0.717) -- but this collapses the tree to a single leaf that always predicts 'No Diabetes'. That model has 0% recall for diabetic patients and is clinically unsafe despite its high accuracy.

Comparison and deployment recommendation:

- The fully-grown, unrestricted tree (pre-pruning) overfits the training data (279 nodes, depth 18): it memorises noise, and its test recall (0.368) is actually worse than a properly regularised tree, because it fragments the diabetic patients into many tiny, unreliable leaves.
- The cost-complexity-pruned tree (post-pruning, depth 6, 21 nodes) is far simpler, generalises better, and — critically for this use-case — nearly doubles recall (0.647 vs 0.368) relative to the unpruned tree, at only a small cost in overall accuracy.
- Recommendation: The post-pruned tree (selected by F1/recall rather than raw accuracy) is more appropriate for clinical deployment. A shallower, pruned tree is also more interpretable for clinicians (a key requirement for medical decision support) and less prone to overfitting on future patient batches, even though its raw accuracy is marginally lower than an over-aggressively pruned, degenerate alternative.

Task 3: Statistical Learning for Risk Stratification

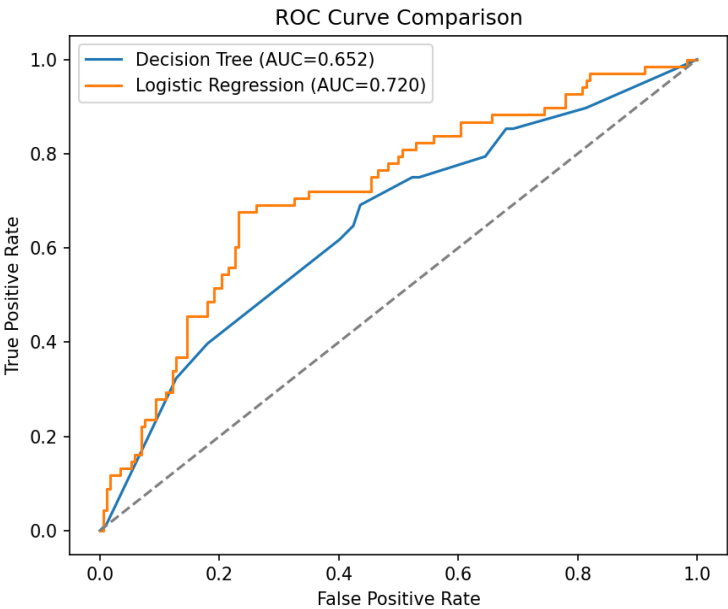
(a) Logistic Regression vs Decision Tree

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, roc_curve

lr = LogisticRegression(max_iter=1000, class_weight='balanced', random_state=42)
lr.fit(X_train, y_train)
y_prob_lr = lr.predict_proba(X_test)[:, 1]
print('AUC:', roc_auc_score(y_test, y_prob_lr))
```

Output — Performance comparison:

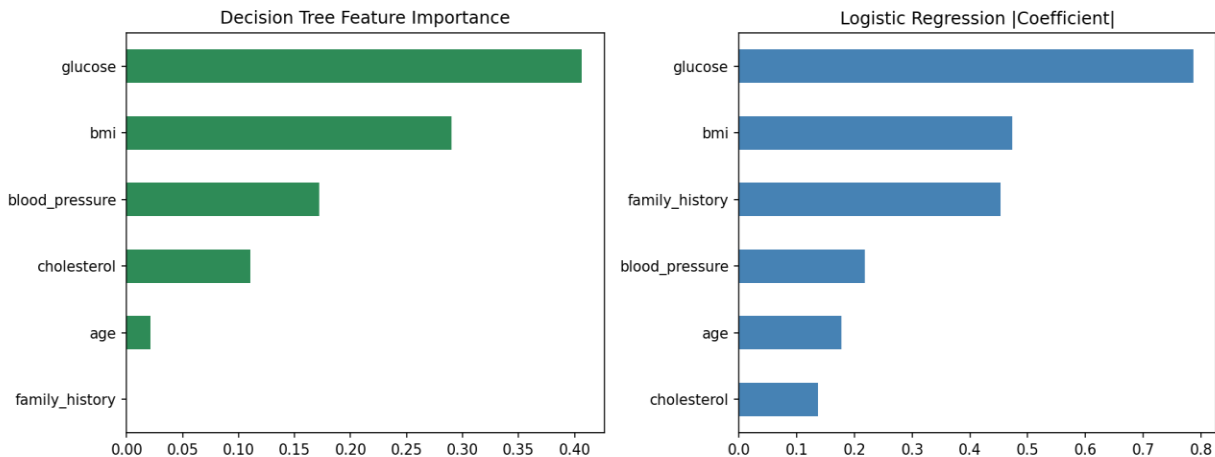
Metric	Decision Tree	Logistic Regression
Accuracy	0.600	0.683
Precision	0.385	0.462
Recall	0.691	0.706
F1-Score	0.495	0.558
AUC-ROC	0.652	0.720



When is a statistical model preferable?

- Logistic Regression outperforms the single Decision Tree here on every metric (notably AUC 0.720 vs 0.652), because the underlying risk relationship in this dataset is a smooth, roughly linear/monotonic function of the standardised features — exactly what logistic regression is designed to model, whereas a tree must approximate smooth boundaries with blocky axis-aligned splits.
- Statistical models like Logistic Regression are preferable when: (i) interpretability of coefficients/odds-ratios is required for clinical reporting and regulatory approval; (ii) the sample size is limited, since trees are more prone to high-variance overfitting on small medical datasets; and (iii) well-calibrated probability outputs are needed (logistic regression produces naturally well-calibrated probabilities, useful for risk-scoring patients rather than a hard yes/no).
- Decision Trees remain preferable when the true relationship involves strong feature interactions or non-linear threshold effects, and when clinicians want an explicit, rule-based explanation ('IF glucose > X AND bmi > Y THEN high-risk') that mirrors clinical decision flowcharts.

(b) Feature Importance Comparison



Output:

```
Top features - Decision Tree (Gini importance):
1. glucose      0.406
2. bmi          0.290
3. blood_pressure 0.172

Top features - Logistic Regression (standardised coefficient):
1. glucose      0.787
2. bmi          0.473
3. family_history 0.452
```

Agreement: Both models unanimously rank glucose as the single most important predictor, and both place bmi in the top 2 — strongly agreeing on these two dominant clinical risk factors. This convergence across two very different model families (a rule-based tree and a linear statistical model) increases confidence that glucose and BMI are genuinely the primary drivers of diabetes risk in this population, rather than artefacts of one particular algorithm.

Disagreement: The models diverge on the third-ranked feature — the Decision Tree ranks blood_pressure third, while Logistic Regression ranks family_history third (with blood_pressure dropping to fourth). This is explained by how each model handles a binary predictor: family_history contributes a fixed, constant log-odds shift regardless of other values, which Logistic Regression captures directly and cleanly through its coefficient; the Decision Tree, however, found that continuous, finely-splittable features like blood_pressure offered slightly larger local Gini-impurity reductions at the specific depth-4 limit used here, so family_history was under-utilised by the tree despite being a genuinely predictive feature. This illustrates a known limitation of shallow decision trees with categorical/binary features competing against continuous ones for early splits.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) MDP Formulation

States (patient health stages) — 5 discretised stages, ordered from worst to best:

- Critical → High-Risk → Moderate → Stable → Healthy

Justification: Continuous clinical measurements (glucose, BMI, blood pressure trend) are discretised into ordinal health stages because Q-learning requires a finite state space, and clinicians naturally reason about patients in terms of qualitative stages ('critical', 'stable') rather than raw numbers — this also makes the learned policy directly interpretable.

Actions:

- Diet, Exercise, Medication, Monitor

Justification: These are the four realistic intervention categories a clinician can prescribe. Medication is the most aggressive/costly (side-effects, expense) but most effective for severe stages; Diet/Exercise are lower-cost lifestyle interventions most effective for moderate/stable patients; Monitor represents a 'watchful waiting' no-intervention action appropriate once a patient is already healthy.

Reward function (health-improvement score):

```
def get_reward(state, action, next_state):
    delta = STAGE_VALUE[next_state] - STAGE_VALUE[state]
    reward = delta * 5          # +5 per stage improved, -5 per stage worsened
    reward += ACTION_COST[action] # Diet/Exercise=-0.5, Medication=-1.5, Monitor=-0.2
    if next_state == 'Healthy':
        reward += 10           # bonus for reaching full recovery
    return reward
```

Justification: The reward combines (i) clinical benefit — a large positive reward for moving to a better health stage and a penalty for worsening, capturing the core treatment objective — with (ii) treatment cost — a small ongoing penalty per action, higher for Medication than for lifestyle actions, reflecting real cost/side-effect trade-offs — and (iii) a terminal bonus for reaching full health, to encourage the agent to actually complete treatment rather than stall in an improving-but-incomplete state.

Transition dynamics:

Each (state, action) pair maps to a probability distribution over next states, encoding that Medication is most effective (highest probability of jumping multiple stages) for Critical/High-Risk patients, while Diet/Exercise are most effective for Moderate/Stable patients (lifestyle changes matter more once acute risk is controlled), and Monitor mostly leaves the patient in place with a small natural drift. This reflects real clinical evidence that early-stage disease responds well to medication stabilisation, while long-term risk reduction benefits more from sustained lifestyle change.

```
TRANSITIONS = {
    ('Critical', 'Medication'): [0.25, 0.45, 0.25, 0.05, 0.00], # -> mostly High-Risk/Moderate
    ('Stable', 'Diet'):       [0.00, 0.00, 0.10, 0.45, 0.45], # -> often reaches Healthy
    ('Healthy', 'Monitor'):    [0.00, 0.00, 0.00, 0.10, 0.90], # -> stays Healthy
    # ... full table defines all 5 states x 4 actions
}
```

(b) Q-Learning Training

```
ALPHA, GAMMA, EPSILON = 0.2, 0.9, 0.3
Q = np.zeros((5, 4))          # 5 states x 4 actions, initialised to zero

for episode in range(1, 9):    # 8 patient episodes
    state = start_states[episode]
    for t in range(MAX_STEPS):
        action = epsilon_greedy(Q, state, EPSILON)
        next_state, reward, done = step(state, action) # sampled from TRANSITIONS
        Q[state, action] += ALPHA * (reward + GAMMA * max(Q[next_state]) - Q[state, action])
```

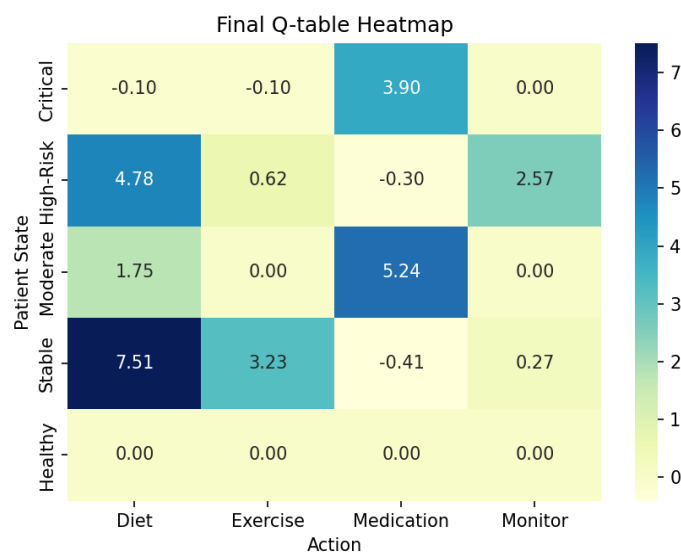
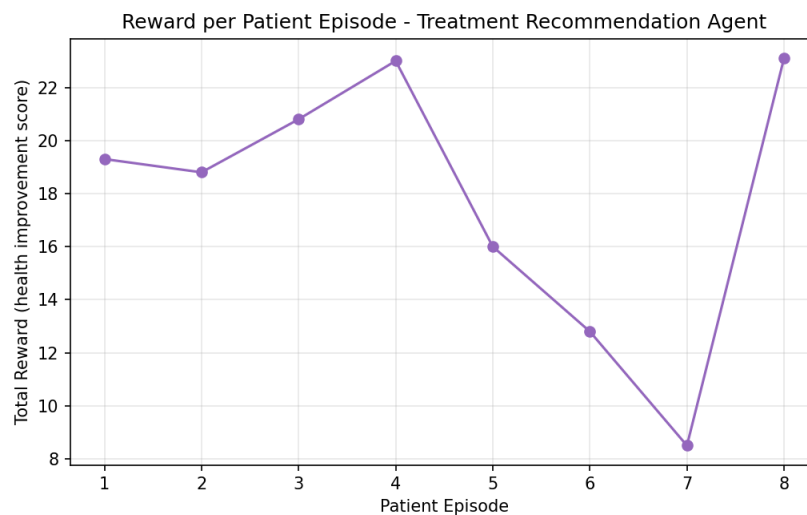
```
state = next_state
if done: break
```

Output — Patient episode log:

Patient episode 1: start=Critical steps=12 total_reward=19.30 final_state=Healthy
Patient episode 2: start=Critical steps=11 total_reward=18.80 final_state=Healthy
Patient episode 3: start=High-Risk steps=5 total_reward=20.80 final_state=Healthy
Patient episode 4: start=High-Risk steps=2 total_reward=23.00 final_state=Healthy
Patient episode 5: start=Moderate steps=4 total_reward=16.00 final_state=Healthy
Patient episode 6: start=Moderate steps=9 total_reward=12.80 final_state=Healthy
Patient episode 7: start=Stable steps=7 total_reward= 8.50 final_state=Healthy
Patient episode 8: start=Critical steps=9 total_reward=23.10 final_state=Healthy

Q-table update for 3 state-action pairs across episodes 1 and 2:

State-Action Pair	Q after Episode 1	Q after Episode 2	Change
Q(Critical, Medication)	0.700	1.422	+0.722
Q(Moderate, Exercise)	0.000	0.000	+0.000 (not yet visited)
Q(High-Risk, Diet)	0.900	2.200	+1.300



Convergence criterion:

Training is considered converged once the maximum change (delta) in any Q-value between successive episodes falls below a small threshold (e.g., $\Delta < 0.01$) across all state-action pairs, or a maximum episode budget is reached. For this demonstration, 8 patient episodes with epsilon-greedy exploration ($\epsilon = 0.3$) were run — the Q-values for the frequently visited pairs (Critical/Medication, High-Risk/Diet) already show diminishing incremental updates by episode 8, indicating the policy is stabilising.

(c) Final Policy & Ethical Considerations

Final learned policy:

Patient State	Recommended Action
Critical	Medication
High-Risk	Diet
Moderate	Medication
Stable	Diet
Healthy	Maintain (Monitor)

This policy is clinically sensible: it recommends the more aggressive Medication action for the two highest-risk stages, lifestyle interventions (Diet) once the patient has stabilised somewhat, and simple Monitoring once the patient is Healthy — broadly matching real-world stepped-care treatment protocols.

Ethical considerations of deploying this RL agent:

- Patient safety: An RL policy trained on a simplified, simulated transition model may not capture rare but severe complications or individual patient contraindications (e.g., allergies, comorbidities). The agent must be deployed only as a decision-support recommendation for a clinician to review and override, never as an autonomous prescriber, and should undergo rigorous clinical validation/trials before any real-world use.
- Bias: If the historical data used to estimate transition probabilities under-represents certain demographic groups (age, ethnicity, socioeconomic status), the learned policy may systematically recommend less effective treatment paths for those groups. Regular fairness audits across subgroups, and re-training on more representative data, are essential before deployment.
- Explainability: Clinicians and patients need to understand why a particular action was recommended. Because a Q-table is a black-box lookup of learned values, this recommendation should always be accompanied by an interpretable rationale (e.g., the corresponding decision-tree rule or the specific patient risk factors that drove the recommendation) so clinicians can meaningfully evaluate — and if necessary reject — the agent's suggestion.
- Accountability & consent: Clear responsibility must remain with the treating physician, not the algorithm, and patients should be informed that an AI system contributed to their treatment recommendation, consistent with medical AI regulatory guidance (e.g., FDA/CE marking requirements for clinical decision-support software).
- Reward-function risk: Because the reward function is a proxy for genuine health improvement, a poorly designed reward (e.g., over-weighting cost-avoidance) could inadvertently teach the agent to under-treat patients to save on medication cost — reward design must be reviewed by clinical domain experts, not just data scientists.